

Reporte de implementación y experimentos

Detección de tráfico de red malicioso mediante clasificación
con desbalance extremo de clases e interpretabilidad

Proyecto Aplicado en Analítica de Datos — Módulo 2

Mariana Pérez
Bryan Rodriguez
Daniela Zúñiga

7 de septiembre de 2026

Resumen

Este reporte documenta la implementación y los experimentos del prototipo de detección de tráfico de red malicioso sobre el conjunto de datos CIC-IDS2017. Se describen: (i) los modelos seleccionados y su correspondencia con los requerimientos aprobados; (ii) las alternativas evaluadas con sus ventajas y desventajas; (iii) la verificación de los supuestos de cada familia de modelos y el tratamiento previo de los datos que cada una exige; (iv) el protocolo de entrenamiento, validación y parametrización, con la justificación de las métricas empleadas; (v) el análisis de resultados y los ajustes implementados a partir de él; y (vi) el estado de implementación del prototipo y el plan para completarlo. El modelo seleccionado alcanza un macro-F1 de 0,975 sobre el conjunto de prueba, manteniendo las falsas alarmas en 0,12 % del tráfico benigno; la estimación conservadora, que no depende del puerto de destino, es de 0,942.

1. Contexto y alcance

Problema: Un centro de operaciones de seguridad (SOC) recibe cientos de miles de *flujos de red* por día. Un flujo es una conversación entre dos computadores resumida en medidas numéricas (cuántos paquetes, de qué tamaño, con qué ritmo, cuánto duró), sin acceso al contenido de la comunicación. Revisarlos manualmente es inviable y las reglas por firma no reconocen ataques nuevos.

Usuario objetivo: Analista de seguridad de un SOC. Conoce redes y tipos de ataque, pero no es científico de datos ni programador, y sobre todo se enfrenta a un volumen de tráfico que ninguna persona puede revisar manualmente. Necesita *priorizar* qué flujos investigar y disponer de una razón que pueda justificar ante terceros.

Datos: CIC-IDS2017 (Canadian Institute for Cybersecurity; Sharafaldin et al., 2018): una semana simulada de tráfico corporativo, con 2.830.743 flujos etiquetados como benignos (80,3 %) o como uno de 14 tipos de ataque (19,7 %). El desbalance es extremo: la clase más frecuente supera a la más rara en una razón de 206.645 a 1.

Encuadre analítico: El problema se aborda como uno de **clasificación con desbalance de clases e interpretabilidad**, no como un problema de seguridad ofensiva.

2. Modelos propuestos y su correspondencia con los requerimientos

2.1. Requerimiento al modelo

Los requerimientos aprobados se agrupan en tres preguntas analíticas, y cada una determina una familia de modelos distinta, donde el Cuadro 1 hace esa correspondencia explícita.

Cuadro 1: Correspondencia entre requerimientos, pregunta analítica y modelo.

Req.	Requerimiento	Pregunta analítica	Modelo que responde
R1, R6, R14	Clasificar el tipo de ataque pese al desbalance, con desempeño accionable	¿De qué tipo es este ataque?	Clasificador multiclase supervisado (11 clases)
R2	Reducir falsas alarmas para disminuir la fatiga de alertas	¿Cuántas alarmas falsas genera?	El mismo multiclase, evaluado por precisión y recall por clase
R3	Explicar en qué se basa el modelo	¿Por qué se marcó esta alerta?	Técnicas de interpretabilidad aplicadas al clasificador: lectura global e individual
R4	Detectar comportamientos anómalos no etiquetados	¿Esto es raro, aunque no sepa qué es?	Detector de anomalías no supervisado
R5	Demostrar valor frente al escenario sin herramienta	¿Cuánto trabajo ahorra?	Comparación contra clasificador trivial y contra revisión manual

2.2. Integración de los tres modelos

La solución no es un modelo sino **tres modelos que se complementan**, porque ninguno cubre solo todo el espacio del problema. La Figura 1 muestra cómo se integran.

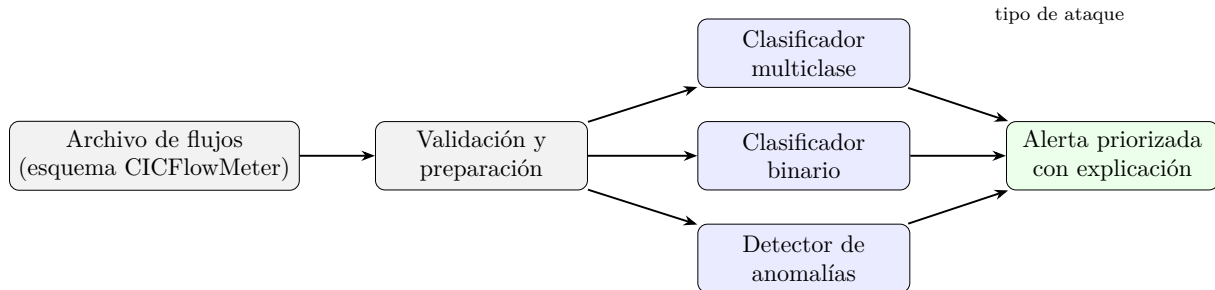


Figura 1: Arquitectura de la solución: los tres modelos operan sobre el mismo flujo preparado y sus salidas se combinan en una única alerta. El multiclase aporta el tipo de ataque; el binario, un veredicto robusto de ataque/no ataque que cubre las clases que el multiclase no pudo aprender; el detector de anomalías, comportamientos raros sin necesidad de etiquetas.

La razón del uso de tres modelos es una limitación de los datos que se documenta más adelante (Sección 7) y son estos dos tipos de ataque: Heartbleed (11 casos) e Infiltration (36 casos), tienen tan pocos casos que no pueden aprenderse como clases propias. El clasificador binario y el detector de anomalías cubren ese vacío desde ángulos distintos.

Qué se despliega y qué queda en el análisis: No todo lo que realizamos hace parte del artefacto (por practicidad), lo separamos en dos categorías para evitar confusión:

- **Modelos desplegados en el artefacto (tres).** El clasificador multiclase y el clasificador binario, ambos *predictivos* (asignan una etiqueta a cada flujo); y el detector de anomalías, *descriptivo*, ya que no predice una etiqueta conocida sino que señala desviaciones respecto del patrón de tráfico normal.

- **Herramientas de análisis e interpretación (no desplegadas como modelos).** La regresión logística, usada como referencia interpretable para verificar el supuesto de linealidad; el Local Outlier Factor, usado como contraste del detector de anomalías; y las técnicas de interpretabilidad (importancia por permutación y SHAP), que son *descriptivas*: no producen una predicción propia sino que explican la del clasificador.

3. Alternativas evaluadas: ventajas y desventajas

Para cada requerimiento se plantearon alternativas y se llevaron a un plan de pruebas comparativo. El Cuadro 2 resume el análisis previo, la Sección 6 reporta cuál ganó y por qué.

Cuadro 2: Alternativas consideradas por requerimiento, con ventajas y desventajas.

Req.	Alternativa	Ventajas	Desventajas
R1	Regresión logística	Coeficientes directamente legibles; entrenamiento simple; sirve de referencia honesta	Supone frontera lineal; exige escalado y control de colinealidad
R1	HistGradient-Boosting	Captura relaciones no lineales; no requiere escalado; muy rápido con datos tabulares grandes	Menos transparente: exige técnicas externas de interpretación
R1	Sin corrección de desbalance	No introduce sesgo artificial; sirve como control	Las clases minoritarias quedan ignoradas
R1	Pesos de clase	Barato; no fabrica datos; ataca la función de pérdida	Si el modelo no puede separar, solo multiplica falsas alarmas
R1	SMOTE (sobremuestreo sintético)	Densifica la clase rara donde ya existe	No crea información nueva; inútil con muy pocos casos
R1	Submuestreo + SMOTE	Reduce costo de cómputo; equilibra el conjunto	Descarta información real; resultó inestable
R4	Isolation Forest	Escala bien; sin etiquetas; umbral interpretable por cuantiles	Ciego a ataques cuya anomalía está en el agregado
R4	Local Outlier Factor	Densidad local en lugar de aislamiento: ve la fuerza bruta contra SSH, que el Isolation Forest no detecta (recall 0,91 frente a 0,00)	Costo de predicción alto; pierde Heartbleed (0,00 frente a 0,89) y los DoS lentos (slowloris 0,00 frente a 0,52); su recall global es un tercio del elegido (0,035 frente a 0,118)
R3	Importancia por permutación	Agnóstica al modelo; se mide sobre datos no vistos	Solo explica el comportamiento global, no la alerta individual
R3	SHAP (TreeExplainer) (Lundberg y Lee, 2017)	Explica cada alerta con fundamento matemático; 2 ms por flujo	Añade dependencias pesadas (≈ 100 MB de memoria)
R3	Desviación vs. rango benigno	Sin dependencias; muy simple de explicar	Describe qué es raro, no qué miró el modelo

En **negrilla**, la alternativa seleccionada para cada requerimiento tras las pruebas comparativas de la Sección 6.

Diseño del plan de pruebas. Las alternativas de R1 se cruzaron en una **matriz de dos ejes** (2 familias de modelo $\times$ 4 técnicas de desbalance = 8 combinaciones), evaluadas todas bajo condiciones idénticas. El diseño no es casual: separar los ejes permite *atribuir cada mejora a su causa* y distinguir si un problema se debe al desbalance o a la capacidad del modelo. Como se verá, esa distinción resultó ser el hallazgo central del proyecto.

4. Verificación de supuestos y tratamiento previo de los datos

Cada familia de modelos impone supuestos distintos sobre los datos. El Cuadro 3 documenta qué se verificó y qué tratamiento se aplicó en consecuencia.

Cuadro 3: Supuestos por familia de modelo, verificación realizada y tratamiento aplicado.

Modelo	Supuesto	Verificación	Tratamiento
Regresión logística	Separabilidad lineal de las clases	Curvas precision-recall por clase: Bot y Web Attack quedan en $AP \approx 0,2$, indicando que no son linealmente separables	Se conserva como referencia interpretable; se añade un modelo no lineal para las clases afectadas
Regresión logística	Variables no redundantes (baja colinealidad)	45 pares con $ r > 0,95$ sobre el conjunto de entrenamiento, agrupados en 13 bloques	Poda a una representante por bloque: $71 \rightarrow 48$ variables
Regresión logística, Isolation Forest	Variables en escala comparable	Rangos heterogéneos (microsegundos frente a conteos)	StandardScaler ajustado <i>dentro</i> del pipeline de validación
HistGradient-Boosting	Ninguno sobre escala o linealidad	Se verifica que el desempeño no dependa del escalado	No se escala; se conserva la escala original
Isolation Forest	Las anomalías son pocas y estructuralmente distintas	Distribución de scores sobre tráfico conocido-benigno	Umbral fijado por cuantiles de esa distribución
Todos	Ausencia de valores no finitos	2.867 filas (0,10 %) con Inf/NaN en dos variables de tasa (divisiones por duración cero)	Eliminadas (1.563 de ellas en este paso; el resto ya había salido con los duplicados), con verificación automática de que no quedan
Todos	Los valores numéricos representan magnitudes	El valor -1 en Init_Win_bytes no es una magnitud sino un código de “no aplica” (≈ 40 % de las filas)	Se convierte en indicador binario y el -1 pasa a 0
Todos	Independencia entre observaciones	330.995 filas duplicadas exactas (11,69 %)	Eliminadas <i>antes</i> de partir en entrenamiento y prueba
Todos	Variables informativas	8 variables constantes (varianza cero)	Eliminadas

4.1. El supuesto de linealidad verificado

La verificación más consecuente fue la de linealidad. La Figura 2 muestra las curvas precision-recall de las dos clases problemáticas con tres configuraciones. Con el modelo lineal, ninguna técnica de desbalance las rescata, es decir, el problema no era la proporción de clases, sino que **sus fronteras de decisión no son lineales**. Al sustituir la familia de modelo (sin ninguna corrección de desbalance) ambas mejoran sustancialmente. Este resultado es lo que motivó el ajuste principal que se realizó en la Sección 7.

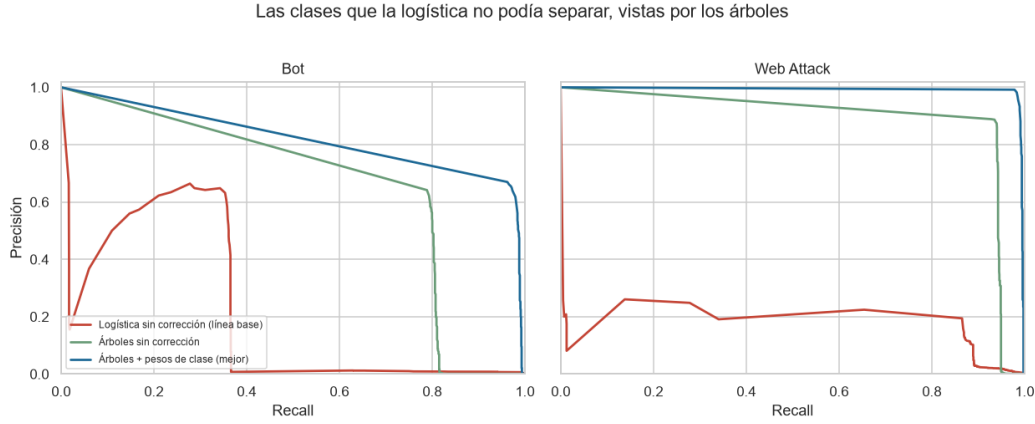


Figura 2: Verificación del supuesto de linealidad. Las curvas se desplazan hacia la esquina superior derecha al cambiar de familia de modelo, no al cambiar la técnica de desbalance. Esta es la evidencia de que el supuesto de separabilidad lineal no se cumple para estas clases.

5. Entrenamiento, validación, parametrización y métricas

5.1. Protocolo experimental

- **Partición única:** 80 % entrenamiento (1.998.462 flujos) y 20 % prueba (499.616), estratificada por clase para preservar las proporciones incluso en las clases más raras. El conjunto de prueba se apartó una sola vez y **no se utilizó** durante todo el desarrollo.
- **Validación cruzada estratificada de 5 particiones** sobre el entrenamiento, para toda decisión: comparación de alternativas, selección de modelo y ajuste del punto de operación. Se reporta media $\pm$ desviación entre particiones.
- **Selección de variables:** 71 $\rightarrow$ 48, decidida *únicamente* con el conjunto de entrenamiento (Cuadro 3).
- **Control de fuga de información:** deduplicación previa a la partición; escalado y remuestreo *dentro* del pipeline de validación, de modo que solo se ajustan con el tramo de entrenamiento de cada partición.
- **Reproducibilidad:** semilla fija (42) en todo el proceso y versiones exactas de dependencias.

Parametrización: El modelo de árboles usa 200 iteraciones y ponderación balanceada de clases. SMOTE eleva a 20.000 casos las clases con menos de ese umbral (igualarlas a la clase mayoritaria habría exigido fabricar ≈ 14 millones de filas sintéticas); el submuestreo reduce la clase benigna a 200.000 casos antes de aplicar SMOTE.

Calibración de hiperparámetros: No se realizó una búsqueda exhaustiva de hiperparámetros de forma deliberada. Con los valores estándar el modelo ya superaba el requerimiento de desempeño por más de tres puntos de macro-F1, y la matriz de la Sección 6 muestra que la variación atribuible a la familia de modelo y a la técnica de desbalance (de 0,543 a 0,970) domina ampliamente cualquier ganancia esperable de un ajuste fino de parámetros. El esfuerzo de calibración se dirigió, en cambio, donde el análisis mostró margen real: el *punto de operación* de la clase Bot (Sección 7, ajuste 4), fijado con datos de entrenamiento exclusivamente. Una búsqueda adicional habría añadido riesgo de sobreajustar al conjunto de validación a cambio de una ganancia marginal.

5.2. Justificación de las métricas

La elección de métricas es clave en problemas con este nivel de desbalance:

- **Exactitud (accuracy): descartada.** Con 83 % de tráfico benigno en el conjunto ya depurado (80,3 % en los datos crudos: la deduplicación redujo más a los ataques que al tráfico normal), un clasificador que responda siempre “normal” alcanza 83 % de exactitud sin detectar un solo ataque. Se verificó empíricamente: ese clasificador trivial obtiene macro-F1 de 0,082 en el problema multiclase.
- **Macro-F1: métrica principal.** Promedia el F1 de todas las clases con igual peso, de modo que una clase con 1.500 casos pesa lo mismo que una con dos millones. Es la métrica adecuada cuando las clases raras son las que importan.
- **Precisión y recall por clase: obligatorios.** El promedio puede ocultar que una clase concreta no se detecta. En el contexto del SOC tienen lectura directa: el recall es la fracción de ataques encontrados; la precisión, la fracción de alarmas que no son falsas (fatiga de alertas).
- **Curvas precision-recall y AP en lugar de ROC/AUC.** La tasa de falsos positivos que usa la curva ROC tiene por denominador la clase mayoritaria: con dos millones de flujos benignos, 40.000 falsas alarmas son apenas un 2 % y la curva luce excelente, mientras la precisión real se desploma. La curva PR expone ese costo. Además, el AP resultó una herramienta *de diagnóstico*: un AP alto con recall bajo indica un problema de umbral, mientras que un AP bajo indica que el modelo no separa la clase.
- **Macro-F1 restringido a clases de ataque:** se reporta junto al global como cifra más exigente, al excluir del promedio la clase benigna.

5.3. Líneas base

Antes de experimentar se fijaron dos tipos de referencia (baseline) contra los cuales medir cualquier mejora posterior (Cuadro 4). La primera es el **clasificador trivial**: un modelo que responde siempre la clase mayoritaria sin mirar los datos. Marca lo que sería el piso absoluto, porque cualquier modelo que no lo supere no estaría aprendiendo nada. La segunda es la **regresión logística sin corrección de desbalance**, que representa la solución más simple y transparente aplicable al problema. Ambas se calcularon con el mismo protocolo de validación cruzada que las alternativas posteriores, de modo que todas las cifras del reporte son comparables entre sí.

Cuadro 4: Líneas base (validación cruzada sobre el conjunto de entrenamiento).

Configuración de referencia	macro-F1
Binaria — clasificador trivial (siempre “normal”)	0,453
Binaria — regresión logística	0,921
Multiclase — clasificador trivial (siempre “benigno”)	0,082
Multiclase — regresión logística	0,701

6. Análisis de resultados

6.1. Comparación de alternativas

El Cuadro 5 presenta las ocho combinaciones evaluadas bajo condiciones idénticas, y la Figura 3 las ordena visualmente.

Cuadro 5: Matriz de alternativas: macro-F1 (media $\pm$ desviación entre las 5 particiones).

Modelo	Sin corrección	Pesos de clase	SMOTE	Submues. + SMOTE
Regresión logística	0,700 $\pm$ 0,014	0,543 $\pm$ 0,004	0,832 $\pm$ 0,005	0,650 $\pm$ 0,002
Árboles (HistGB)	0,948 $\pm$ 0,008	0,970 $\pm$ 0,002	0,966 $\pm$ 0,003	0,937 $\pm$ 0,046

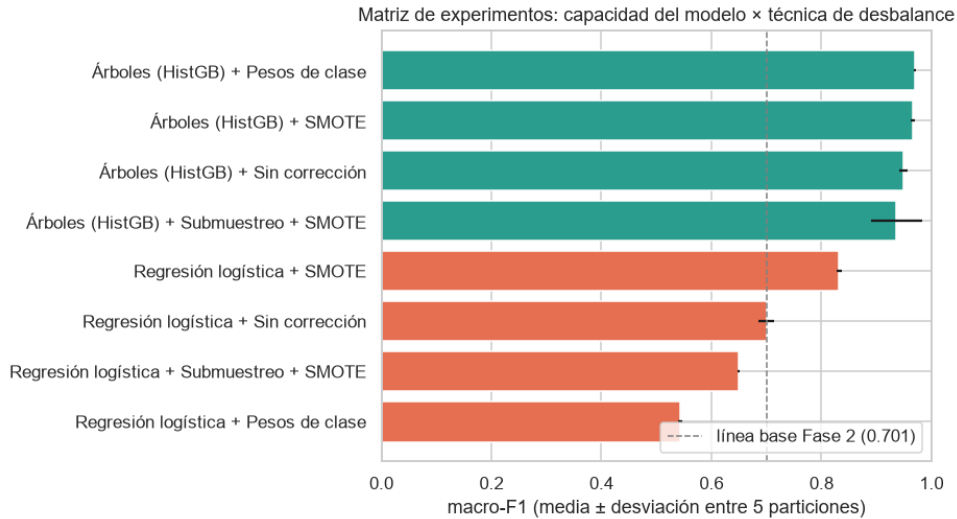


Figura 3: Las ocho combinaciones ordenadas por desempeño. Todas las configuraciones basadas en árboles superan a todas las lineales.

Encontramos tres hallazgos a partir de la matriz:

1. **La capacidad del modelo pesó más que la técnica de desbalance.** La peor configuración de árboles (0,937) supera a la mejor configuración lineal (0,832). Esto nos indicó que más que “falta corrección de desbalance” era “falta capacidad de representación”.
2. **Reponderar sin capacidad es contraproducente.** Los pesos de clase *hunden* a la regresión logística (0,700 $\rightarrow$ 0,543): elevan el recall de las clases minoritarias a $\approx 0,99$ pero con precisión de 0,02–0,03, lo que equivale a ≈ 264.000 flujos benignos marcados como ataque. Con un modelo capaz, la misma técnica es la mejor (0,948 $\rightarrow$ 0,970).
3. **La estabilidad importa tanto como la media.** La combinación submuestreo + SMOTE se descartó no por su media (0,937) sino por su desviación (0,046): una de las particiones cayó a 0,846.

6.2. Modelo seleccionado para cada requerimiento

Con la evidencia anterior se fijó qué modelo responde a cada requerimiento aprobado del prototipo. Los códigos son los mismos del Cuadro 1, y el enunciado de cada uno se repite aquí para facilitar la lectura.

- **R1, R6 y R14** — clasificar el tipo de ataque pese al desbalance, con desempeño accionable: **HistGradientBoosting con pesos de clase**.
- **R3** — explicar en qué se basa el modelo: **importancia por permutación** para la lectura global y **SHAP** para la explicación de cada alerta individual. La regresión logística se conserva como lectura auxiliar del análisis, no como componente del artefacto.

- **R4** — detectar comportamientos anómalos no etiquetados: **Isolation Forest** entrenado únicamente con tráfico benigno. El Local Outlier Factor se documenta como contraste y no se despliega.

7. Ajustes implementados a partir de los resultados

El análisis condujo a cinco ajustes, tres de ellos sobre la propia definición del problema de analítica:

1. Ajuste al esquema de solución: de un modelo a tres. Consecuencia directa del hallazgo de linealidad y de la escasez de dos clases. Ningún modelo único cubre el problema completo (Figura 1).

2. Ajuste al problema de analítica: agrupación de clases: Los tres ataques web (1.470, 652 y 21 casos) se agrupan en la familia “Web Attack”. La agrupación es la natural del dominio: los tres atacan la aplicación web.

3. Ajuste al problema de analítica: exclusión honesta de clases: Heartbleed (11 casos) e Infiltration (36) se excluyen del problema multiclase, ya que con validación cruzada de 5 particiones aportarían dos a siete casos por partición y cualquier métrica por clase sería ruido. Se evalúan en el nivel binario y con el detector de anomalías, y se reportan siempre acompañadas de su tamaño de muestra. Declarar la limitación es preferible a publicar una cifra que el dato realmente no sostiene.

4. Ajuste al punto de operación: umbral para la clase Bot. La clase Bot alcanzaba precisión de 0,614 con recall de 0,980, es decir cuatro de cada diez alarmas eran falsas. Su AP de 0,91 indicaba margen para intercambiar recall por precisión. Se fijó, *con datos de entrenamiento exclusivamente*, la regla de emitir la alarma de Bot solo cuando la probabilidad supere 0,999. Resultado sobre el conjunto de prueba: precisión 0,947 con recall 0,728.

5. Verificación de validez: la prueba del puerto de destino. La segunda variable más influyente resultó ser el puerto del servicio contactado, un *identificador* y no un comportamiento: en el conjunto de datos cada ataque usa siempre su puerto canónico, algo que en una red real no necesariamente tiene que pasar. Se reentrenó el modelo sin esa variable (Figura 4): el desempeño pasó de 0,970 a 0,950 en validación cruzada y de 0,967 a 0,942 en prueba. La caída es pequeña y el recall se mantiene en todas las clases, lo que indica que el modelo aprende comportamiento, y el costo se concentra en la precisión de las dos clases difíciles. Por honestidad metodológica, ambas cifras se reportan juntas y la variante sin puerto se declara como la **estimación conservadora**.

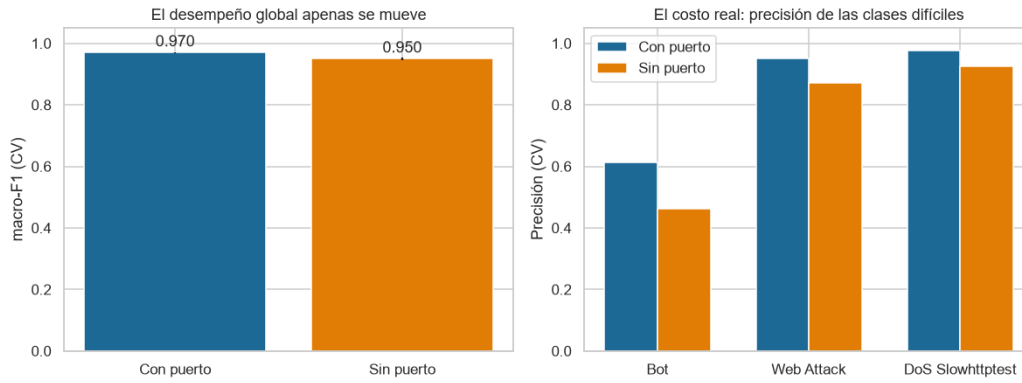


Figura 4: Prueba del puerto de destino. El desempeño global apenas se mueve al retirar el identificador; el costo se localiza en la precisión de las clases difíciles.

7.1. Resultados sobre el conjunto de prueba

Con el modelo ya definido, se utilizó el conjunto de prueba por primera y única vez (para tener los resultados como una medición completamente limpia).

Cuadro 6: Evaluación final sobre el conjunto de prueba (499.616 flujos).

Configuración	macro-F1	Solo ataques
Modelo final (con puerto + umbral de Bot)	0,975	0,972
Con puerto, regla estándar	0,967	0,964
Estimación conservadora (sin el puerto)	0,942	0,937

Validez del protocolo: Las cifras de prueba coinciden con las de validación cruzada ($0,970 \pm 0,002$ frente a $0,967$ con puerto; $0,950 \pm 0,004$ frente a $0,942$ sin él). Esa coincidencia es la evidencia de que no hubo sobreajuste ni fuga de información.

Cumplimiento de requerimientos: R6 exigía $\text{macro-F1} \geq 0,94$: se obtuvo $0,975$ ($0,942$ en la estimación conservadora). R2 exigía falsas alarmas $\leq 0,2\%$ sobre tráfico benigno: se obtuvo $0,12\%$. R5 exigía superar sustancialmente el clasificador trivial: $0,975$ frente a $0,082$. La segunda parte de R5, la reducción de la carga de revisión, se materializa en el panel de resumen del tablero, que la calcula sobre el archivo que carga el usuario.

Detección no supervisada (R4): El Isolation Forest, entrenado únicamente con tráfico benigno, detecta 89% de los flujos de Heartbleed, 52% de DoS slowloris y 48% de Infiltration (justamente las familias que el modelo supervisado no pudo aprender) con un presupuesto de 1% de falsas alarmas. Su recall global es de $11,8\%$, y se documenta explícitamente su punto ciego: es incapaz de detectar ataques camuflados (fuerza bruta, escaneo de puertos, botnet), porque en ellos cada flujo individual parece normal y lo anómalo reside precisamente en el agregado. El recall superior a $0,4$ en las familias estructuralmente raras se cumple como exigía el R4

Deriva temporal (R11): La tasa de falsas alarmas del detector se mantuvo en torno al 1% previsto casi todos los días, con una excepción documentada: un tramo del viernes alcanzó $9,6\%$. Se reporta como señal de que un despliegue real exigiría recalibración periódica.

8. Estado de implementación y plan para completar el prototipo

8.1. Componentes ya implementados

A la fecha de este reporte están implementados y verificados:

- **Modelos serializados** (1,3 MB en total) con sus metadatos, y verificación de determinismo: los umbrales reproducidos coinciden decimal a decimal con los del análisis.
- **Tablero web** (Streamlit) con seis pantallas funcionales: resumen, clasificación, detección de anomalías, interpretabilidad, cola de alertas y reportes. Incluye validación de archivos de entrada, filtros, exportación en CSV e historial de sesión. La interfaz se seguirá refinando antes de la entrega final.
- **Archivos de demostración:** dos muestras del conjunto de prueba. Una con 499 flujos con 39,9 % de ataque y la otra con 500 flujos con 5,0 %, esta última tiene una proporción más realista, y se crearon para probar la herramienta con archivos mucho más manejables.
- **Pruebas automatizadas** (25 casos) sobre validación de entradas, clasificación, umbrales y explicaciones. Los scripts se ejecutan fuera del repositorio, incorporarlos a él está pendiente.
- **Verificación de desempeño (R9):** 50.000 flujos clasificados en menos de un segundo (0,7–0,8s en mediciones repetidas) con un pico de memoria cercano a 390 MB, frente a la promesa de 30 segundos y 1 GB disponible.
- **Licencias:** todas las dependencias son de código abierto con licencias permisivas (BSD, MIT, Apache 2.0, PSF), compatibles con uso académico.

8.2. Componentes pendientes y plan

El Cuadro 7 lista los componentes que restan, con el requerimiento que los motiva. Todos ellos se completan como parte de la entrega final del proyecto, antes de la evaluación del prototipo y de la presentación ejecutiva.

Cuadro 7: Plan para completar la implementación del prototipo.

Pendiente	Descripción	Req.
Despliegue en URL pública	Publicar el tablero en alojamiento gratuito con la versión de Python con la que se congeló el entorno; verificar el tiempo de reanudación tras inactividad	R10, R22
Manual de usuario	Qué hace la herramienta, sus limitaciones y advertencias, requisitos previos y al menos tres casos de uso paso a paso	Entregable
Rúbrica de pruebas diligenciada	Tabla de requerimientos con evidencia de cumplimiento, ajustes implementados y justificación de lo no satisfecho	Entregable
Tabla de falsas alarmas por día en el tablero	Los datos ya están calculados; falta exponerlos en la interfaz	R11
Prueba de usabilidad con usuario ajeno	Verificar que un usuario ajeno completa cargar → revisar → exportar sin asistencia	R20
Scripts de prueba en el repositorio	Los 25 casos automatizados se ejecutan hoy fuera del repositorio; incorporarlos los hace verificables por un tercero	Entregable
Presentación ejecutiva	Siete minutos: problema, esquema de solución, demostración y propuesta de valor	Entregable

8.3. Riesgos identificados

- **Sensibilidad de versión de los artefactos:** Los modelos serializados dependen de la versión exacta de la librería con que se generaron. Mitigación: versiones congeladas, verificación al arrancar y mensaje de advertencia explícito.

- **Limitaciones del alojamiento gratuito:** No garantiza persistencia entre reinicios ni tiempos de reanudación acotados. Mitigación: el historial se declara como de sesión, no persistente.
- **Calidad del conjunto de datos:** Encontramos papers que documentan errores de etiquetado en CIC-IDS2017 (Engelen et al., 2021; Rosay et al., 2022; Lanvin et al., 2023). Se declara como limitación y afecta principalmente al detector no supervisado, y se mitiga fijando el umbral por cuantiles asumiendo explícitamente cierta contaminación.

9. Conclusiones

1. El tipo de ataque puede clasificarse con alta calidad pese al desbalance extremo: macro-F1 de 0,975 sobre el conjunto de prueba, y 0,942 en la estimación conservadora que no depende del puerto de destino.
2. **El diagnóstico correcto importa más que la técnica.** Las clases que parecían imposibles no necesitaban más remuestreo sino un modelo capaz de representar fronteras no lineales. Reponderar sin capacidad multiplica las falsas alarmas.
3. Los ataques se distinguen por *comportamiento* (duración, ritmo entre paquetes, tamaños) y no solo por el puerto: retirar el único identificador cuesta entre dos y tres puntos de macro-F1.
4. El modelo no supervisado es un complemento, no un sustituto: detecta sin etiquetas justamente las familias que el supervisado no puede aprender, pero es ciego a los ataques camuflados.
5. El prototipo está implementado y verificado en sus componentes centrales; el plan de la Sección 8 cierra lo pendiente antes de la evaluación y la presentación ejecutiva.

Referencias

- Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). *Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization*. 4th International Conference on Information Systems Security and Privacy (ICISSP), Portugal, enero de 2018.
- Engelen, G., Rimmer, V., & Joosen, W. (2021). *Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study*. IEEE Security and Privacy Workshops.
- Rosay, A., et al. (2022). *Network Intrusion Detection: A Comprehensive Analysis of CIC-IDS2017*. ICISSP.
- Lanvin, M., et al. (2023). *Errors in the CICIDS2017 Dataset and the Significant Differences in Detection Performances It Makes*. CRiSIS 2022.
- Lundberg, S. M., & Lee, S. I. (2017). *A Unified Approach to Interpreting Model Predictions*. Advances in Neural Information Processing Systems (NeurIPS).